

Technical Report on Design Patterns and Microservices Architecture in mimoApp

Kevin Estiven Lozano Duarte
20221020152
Engineer System Student
University Distrital Francisco Jose De Caladas

CONTENTS

I	Introduction	1
II	Design Patterns Overview	1
II-A	Singleton Pattern	1
II-B	Factory Pattern	1
II-C	Adapter Pattern	1
II-D	Decorator Pattern	1
III	Microservices Architecture	1
III-A	Service Decomposition	1
III-B	Communication Between Services	2
IV	Implementation of Patterns in Microservices	2
IV-A	Repository Pattern	2
V	Scalability and Deployment	2
VI	Conclusions and Future Work	2
	References	2

LIST OF FIGURES

1	Microservices Architecture of mimoApp	2
---	---	---

LIST OF TABLES

Technical Report on Design Patterns and Microservices Architecture in mimoApp

Abstract—This technical report explores the design patterns and microservices architecture implemented in mimoApp, highlighting the strategic use of software engineering paradigms for scalability, maintainability, and modularity. The report examines the adoption of patterns such as Singleton, Factory, Repository, Observer, Adapter, and Decorator, along with the application of the microservices architecture style using FastAPI and Docker. Practical examples illustrate best practices in modern backend development. Comprehensive references support the design decisions and align with the principles of Modelos de Programación.

I. INTRODUCTION

mimoApp is a web application leveraging modern software architecture patterns to deliver scalable and maintainable educational services. This report examines the strategic use of design patterns and microservices architecture to enhance system modularity and flexibility. The study is grounded in established software engineering principles [1], [2].

II. DESIGN PATTERNS OVERVIEW

Design patterns provide reusable solutions to common software design problems, enhancing maintainability and scalability. mimoApp utilizes several patterns, including:

A. Singleton Pattern

The Singleton Pattern ensures a class has only one instance, optimizing shared resources like database connections.

Listing 1. Singleton Pattern for Database Connection

```
1 class Database:
2     _instance = None
3     def __new__(cls):
4         if cls._instance is None:
5             cls._instance = super(Database,
6                                   cls).__new__(cls)
7             cls._instance.engine =
8                 create_engine("postgresql://
9                               user:pass@localhost/dbname")
10            return cls._instance
```

B. Factory Pattern

The Factory Pattern provides an interface for creating objects without specifying their concrete classes.

Listing 2. Factory Pattern for Notification System

```
1 class NotificationFactory:
2     def create_notification(self,
3                             notification_type):
4         if notification_type == "email":
5             return EmailNotification()
6         elif notification_type == "sms":
7             return SMSNotification()
```

C. Adapter Pattern

The Adapter Pattern allows incompatible interfaces to work together. In mimoApp, it integrates external APIs with the internal system.

Listing 3. Adapter Pattern for External API Integration

```
class ExternalPaymentAPI:
    def process_payment(self, amount):
        print(f"Processing payment of {amount}
              using external API")

class PaymentAdapter:
    def __init__(self, external_api):
        self.external_api = external_api

    def pay(self, amount):
        self.external_api.process_payment(
            amount)
```

This pattern decouples the application from third-party APIs, enhancing maintainability.

D. Decorator Pattern

The Decorator Pattern adds behavior to objects dynamically. In mimoApp, it's used for logging and authentication in FastAPI endpoints.

Listing 4. Decorator Pattern for Logging and Authentication

```
1 def authenticate(func):
2     def wrapper(*args, **kwargs):
3         user = get_current_user()
4         if not user:
5             raise HTTPException(status_code
6                                 =401, detail="Unauthorized")
7         return func(*args, **kwargs)
8     return wrapper
9
10 @authenticate
11 def get_user_data(user_id):
12     return {"user_id": user_id, "data": "User
13         data here"}
```

This enhances code reusability and modularity, aligning with best practices [1].

III. MICROSERVICES ARCHITECTURE

mimoApp uses microservices architecture to decompose the application into loosely coupled services for independent deployment and scaling [3].

A. Service Decomposition

The system is divided into:

- **User Service:** Authentication and profile management.

- **Course Service:** Course creation and content delivery.
- **Notification Service:** Multi-channel notifications.
- **Payment Service:** Payment processing and transaction history.

B. Communication Between Services

Communication is achieved using RESTful APIs with FastAPI and RabbitMQ for event-driven messaging.

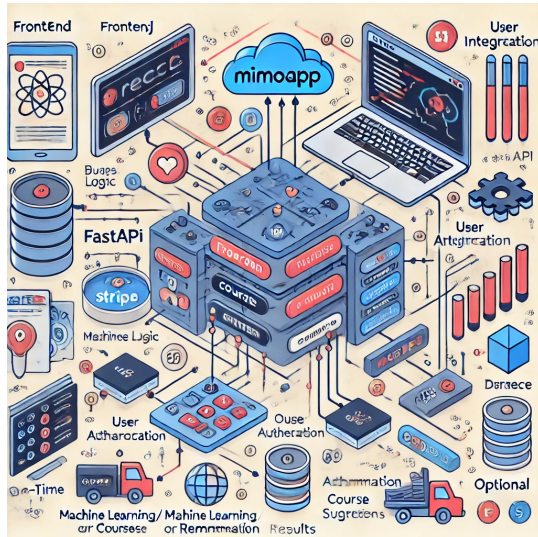


Fig. 1. Microservices Architecture of mimoApp

REFERENCES

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [3] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015.
- [4] C. Richardson, *Microservices Patterns: With examples in Java*. Manning Publications, 2018.
- [5] Kubernetes, *Kubernetes Documentation*. Available: <https://kubernetes.io/docs/>.

IV. IMPLEMENTATION OF PATTERNS IN MICROSERVICES

A. Repository Pattern

The Repository Pattern abstracts the data access layer, promoting loose coupling between the application and the database.

Listing 5. Repository Pattern for User Data Access

```

1 class UserRepository:
2     def __init__(self, db_session):
3         self.db_session = db_session
4
5     def get_user_by_id(self, user_id):
6         return self.db_session.query(User).
           filter(User.id == user_id).first()

```

V. SCALABILITY AND DEPLOYMENT

mimoApp leverages Docker for containerization, ensuring consistent deployment across environments. Kubernetes handles orchestration, supporting high availability and auto-scaling [5].

VI. CONCLUSIONS AND FUTURE WORK

The strategic use of design patterns and microservices architecture in mimoApp enhances scalability, maintainability, and modularity. Future work includes:

- Integrating event sourcing for state management.
- Exploring serverless computing for cost-effective scaling.
- Implementing GraphQL for flexible API queries.